

TechSapo システムガイド

Wall-Bounce 壁打ちシステム統合文書

TechSapo Documentation Team

2025-10-07

Contents

1	TechSapo Wall-Bounce システム	2
1.1	システム概要	2
1.2	アーキテクチャ	3
1.3	環境セットアップ	4
1.4	LLM CLI 使用方法	6
1.5	API リファレンス	7
1.6	GPT-5 vs GPT-5 Codex	9
1.7	品質保証メカニズム	11
1.8	トラブルシューティング	13
1.9	セキュリティ考慮事項	16
1.10	デプロイメント	18
1.11	まとめ	20

1 TechSapo Wall-Bounce システム

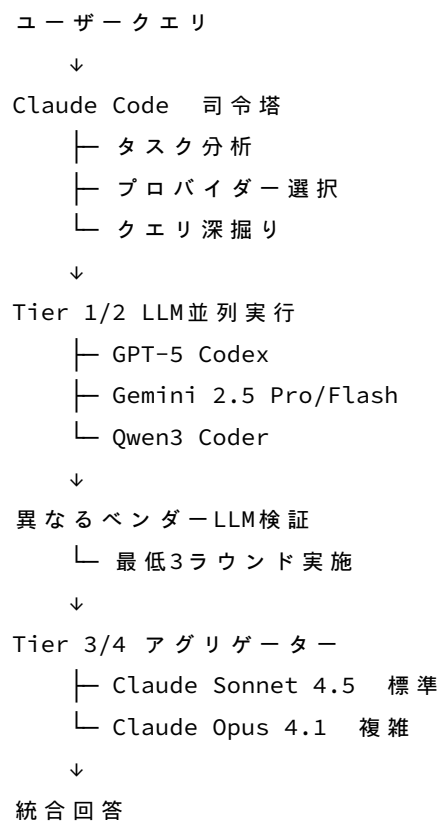
1.1 システム概要

TechSapo は、複数の LLM (Large Language Model) を協調させて高品質な回答を生成する壁打ちシステムです。

1.1.1 コアコンセプト

1. 最低 2 つの LLM で分析 絶対原則
2. 異なるベンダーの LLM で相互検証
3. コンセンサスによる品質保証
4. 階層的プロバイダー選択 Tier 0-4

1.1.2 基本フロー



1.2 アーキテクチャ

1.2.1 Tier 階層構成

Tier 0: ドキュメント参照 非 **LLM** - Context7 MCP - Stash

Tier 1: 高速・コスト効率 - Gemini 2.5 Pro 高品質分析、Deep Think 対応 - Gemini 2.5 Flash 高速分析、最良コスト - Gemini 2.5 Pro Deep Think 深い推論モード

Tier 2: 専門特化 - GPT-5 Codex コーディング特化、適応推論 - GPT-5 汎用推論・分析 - Qwen3 Coder 480B MoE、agentic coding

Tier 3: デフォルト集約 - Claude Sonnet 4.5 高速統合、標準アグリゲーター

Tier 4: 複雑タスク集約 - Claude Opus 4.1 深い分析、複雑統合

1.2.2 タスク別プロバイダーマッピング

1.2.2.1 コーディングタスク

- **Primary:** GPT-5 Codex + Qwen3 Coder
- **Validation:** Gemini 2.5 Flash 異なるベンダー
- **Aggregator:** Sonnet 4.5

1.2.2.2 分析タスク

- **Primary:** GPT-5 + Gemini 2.5 Pro
- **Validation:** Gemini 2.5 Flash
- **Aggregator:** Sonnet 4.5

1.2.2.3 複雑タスク

- **Primary:** Gemini 2.5 Pro (Deep Think) + GPT-5 + Sonnet 4.5
- **Aggregator:** Opus 4.1

1.2.2.4 基本タスク

- **Primary:** Gemini 2.5 Flash + GPT-5
- **Aggregator:** Sonnet 4.5

1.3 環境セットアップ

1.3.1 必須環境変数

```
# Hugging Face 必須
HUGGINGFACE_API_KEY=hf_xxx

# OpenRouter 必須 - Qwen3 Coder
OPENROUTER_API_KEY=sk-or-xxx

# Upstash Redis 必須
UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=xxx

# AWS Secrets Manager 本番推奨
AWS_REGION=ap-northeast-1
AWS_SECRET_NAME=techsapo/production

# Google Cloud Gemini 用
GOOGLE_APPLICATION_CREDENTIALS=/path/to/credentials.json

# OpenAI GPT-5 Codex - Codex MCP 経由
OPENAI_API_KEY=sk-xxx

# Anthropic デフォルト無効
ANTHROPIC_API_ENABLED=false
ANTHROPIC_API_KEY=sk-ant-xxx
```

1.3.2 クイックスタート

```
# リポジトリ移動
cd /ai/prj/techdev

# 依存関係インストール
npm install

# 環境変数設定
cp .env.example .env
# .env を編集して API キーを設定

# ビルド
npm run build

# 開発サーバー起動
```

```
npm run dev
```

```
# ヘルスチェック
```

```
curl http://localhost:5000/health
```

1.3.3 主要コマンド

```
# 開発
```

```
npm run dev # 開発サーバー ポート 5000
```

```
npm start # 本番サーバー ポート 8443
```

```
npm run build # TypeScript ビルド
```

```
# テスト
```

```
npm test # 全テスト 10 分タイムアウト
```

```
npm run test:unit # ユニットテストのみ
```

```
npm run test:integration # 統合テストのみ
```

```
npm test -- tests/path/to/file.ts # 特定ファイル
```

```
# リント
```

```
npm run lint
```

```
# MCP Services
```

```
npm run cipher-mcp # Cipher MCP サーバー起動
```

```
claude mcp list # MCP 状態確認
```

1.4 LLM CLI 使用方法

1.4.1 GPT-5 Codex via Codex MCP

```
# 基本実行
codex exec --model gpt-5-codex "Write TypeScript validator"

# 設定付き実行
codex exec --model gpt-5-codex \
  -c temperature=0.7 \
  -c max_tokens=4096 \
  --sandbox read-only \
  "Fix authentication bug"
```

プロンプトベストプラクティス:

- 簡潔で具体的 Less is more
- `codex exec --model gpt-5-codex "Implement OAuth2 flow"`
- 冗長すぎる 不要

1.4.2 Gemini 2.5 via Gemini CLI

```
# デフォルトモデル gemini-2.5-flash
gemini "Analyze system architecture"

# モデル指定
gemini --model gemini-2.5-pro "Complex analysis task"

# Deep Think モード 自動
gemini --model gemini-2.5-pro "Solve mathematical proof"
```

1.5 API リファレンス

1.5.1 POST /api/v1/wall-bounce/analyze

複数 LLM による協調分析を実行。

リクエスト:

```
{
  "prompt": "Analyze authentication flow",
  "domain": "analysis",
  "minProviders": 2,
  "maxRounds": 5,
  "taskType": "analysis"
}
```

レスポンス:

```
{
  "success": true,
  "result": {
    "content": " 統合された分析結果...",
    "confidence": 0.85,
    "consensus": 0.78,
    "providersUsed": ["gpt-5", "gemini-2.5-pro", "sonnet-4.5"],
    "aggregator": "sonnet-4.5",
    "rounds": 3
  },
  "metadata": {
    "totalLatencyMs": 8500,
    "estimatedCost": 0.024
  }
}
```

1.5.2 POST /api/v1/wall-bounce-sse

ストリーミング応答 Server-Sent Events 。

SSE イベント:

```
event: provider_start
data: {"provider":"gpt-5","round":1}
```

```
event: provider_result
data: {"provider":"gpt-5","content":"..."}
```

```
event: consensus_check
```

```
data: {"consensus":0.75,"threshold":0.6}
```

```
event: complete
```

```
data: {"result":{...}}
```

1.5.3 ヘルスチェックエンドポイント

```
# システムヘルス
```

```
curl http://localhost:8443/health
```

```
# LLM プロバイダーヘルス
```

```
curl http://localhost:8443/api/v1/llm-health
```

```
# MCP ヘルス
```

```
curl http://localhost:8443/api/v1/mcp-health
```


1.6 GPT-5 vs GPT-5 Codex

1.6.1 モデル比較

項目	GPT-5 標準	GPT-5 Codex
特化分野	汎用分析・推論	コーディング
Reasoning	詳細な多段階推論	Adaptive 適応推論
Thinking 詳細度	非常に詳細	ミニマル
コード生成	可能	最適化済み
分析品質	詳細・多角的	簡潔
応答速度	中速	高速

1.6.2 使い分けガイド

GPT-5 標準 を使うべき場合:

```
# 詳細な分析
codex exec --model gpt-5 "Analyze system trade-offs"

# 文章生成
codex exec --model gpt-5 "Write API specification"

# 複雑推論
codex exec --model gpt-5 "Evaluate migration strategy"
```

GPT-5 Codex を使うべき場合:

```
# コーディング
codex exec --model gpt-5-codex "Implement OAuth2 flow"

# デバッグ
codex exec --model gpt-5-codex "Fix: Cannot read property 'map'"

# コードレビュー
codex exec --model gpt-5-codex "Review for security issues"
```

1.6.3 実証結果

コーディングタスク → GPT-5 Codex 自動選択

- Model: gpt-5-codex
- Thinking: ミニマル Drafting のみ
- Output: 即座に高品質コード生成

分析タスク → GPT-5 標準 自動選択

- Model: gpt-5
- Thinking: 多段階 Structuring , Analyzing , Summarizing
- Output: 詳細なセキュリティ分析

1.7 品質保証メカニズム

1.7.1 コンセンサス計算

複数 LLM 応答の類似度を計算し、合意度を評価。

計算式:

$$\text{Consensus} = \Sigma(\text{similarity}(\text{response_i}, \text{response_j})) / n$$

閾値:

- **Consensus 0.6:** 合格
- **Consensus < 0.6:** 追加ラウンド実行

1.7.2 信頼度スコア

各 LLM 応答の信頼度を評価。

評価基準:

- 応答の具体性
- 構造化の度合い
- 専門用語の適切な使用
- 論理的整合性

閾値:

- **Confidence 0.7:** 高品質
- **0.5 Confidence < 0.7:** 要検証
- **Confidence < 0.5:** 再生成

1.7.3 Quorum k 値

最小必要プロバイダー数の動的決定。

基本ルール:

- コーディング: k=2 GPT-5 Codex + Qwen3
- 分析: k=2 GPT-5 + Gemini 2.5 Pro
- 複雑: k=3 Gemini Deep Think + GPT-5 + Sonnet 4.5
- クリティカル: k=4 全 Tier 動員

1.7.4 Early Stopping

Quorum 達成時に即座に終了し、不要な LLM 呼び出しを回避。

条件:

IF (consensus $\geq$ 0.7 AND confidence $\geq$ 0.7 AND providers $\geq$ k):

 STOP and aggregate

ELSE:

 Continue to next round (max 6)

1.8 トラブルシューティング

1.8.1 一般的な問題

1.8.1.1 Redis 接続エラー 症状:

Error: Redis connection failed

解決:

```
# 環境変数確認
echo $UPSTASH_REDIS_REST_URL
echo $UPSTASH_REDIS_REST_TOKEN

# .env ファイル確認
cat .env | grep UPSTASH
```

1.8.1.2 MCP ロックファイル問題 症状:

Error: MCP server already running

解決:

```
# ロックファイル削除
rm /tmp/mcp-*.lock

# MCP 状態確認
claude mcp list
```

1.8.1.3 Gemini API クォータ超過 症状:

ApiError: 429 - You exceeded your current quota

解決:

- Gemini 2.5 Pro にアップグレード より高いクォータ
- リクエスト間隔を調整
- バックオフ・リトライ実装済み 自動

1.8.1.4 GPT-5 Codex 接続失敗 症状:

Error: Codex MCP not responding

解決:

```
# Codex MCP 確認
codex exec --model gpt-5-codex "test"

# OPENAI_API_KEY 確認
echo $OPENAI_API_KEY
```

1.8.2 ヘルスチェック手順

```
# 1. システムヘルス
curl http://localhost:8443/health
# 期待: {"status":"ok"}

# 2. LLM プロバイダーヘルス
curl http://localhost:8443/api/v1/llm-health | jq
# 期待: 全プロバイダー "available": true

# 3. MCP ヘルス
curl http://localhost:8443/api/v1/mcp-health | jq
# 期待: Codex, Context7, Cipher が "connected": true

# 4. Redis 接続
curl http://localhost:8443/api/v1/redis-health
# 期待: {"connected":true}
```

1.8.3 ログ確認

```
# アプリケーションログ
tail -f logs/app.log

# エラーログ
tail -f logs/app-error.log

# IT 統合ログ
tail -f logs/it-unified.log

# systemd ログ 本番
sudo journalctl -u techsapo -f
```

1.8.4 パフォーマンス問題

1.8.4.1 レスポンスが遅い 原因:

- Wall-Bounce ラウンド数が多い 6 ラウンド

- 複雑なプロンプト
- LLM プロバイダー側のレイテンシ

解決:

- minProviders 削減 2→1
- maxRounds 削減 6→3
- taskType= basic を指定 高速モード

1.8.4.2 メモリ使用量が高い 原因:

- 大量の並列リクエスト
- Redis キャッシュ肥大化

解決:

```
# Redis キャッシュクリア
redis-cli FLUSHDB

# Node.js メモリ制限
NODE_OPTIONS="--max-old-space-size=2048" npm start
```

1.9 セキュリティ考慮事項

1.9.1 API キー管理

開発環境:

- .env ファイルで管理
- .gitignore で除外

本番環境:

- AWS Secrets Manager 推奨
- 環境変数での直接設定は非推奨

1.9.2 Anthropic API 制限

デフォルト: 無効

有効化方法:

```
# 環境変数で明示的に有効化
ANTHROPIC_API_ENABLED=true
ANTHROPIC_API_KEY=sk-ant-xxx
```

理由:

- コスト管理
- アクセス制御
- 意図しない使用の防止

1.9.3 PII 保護

自動検出・マスキング:

- メールアドレス
- 電話番号
- クレジットカード番号
- SSN/マイナンバー

監査ログ:

- 全リクエスト・レスポンスを記録
- PII 検出イベントを記録
- /audit/techdev/に保存

1.9.4 レート制限

プロバイダー別制限:

- Gemini: 60 req/min, 1M tokens/min
- OpenAI: 500 req/min, 200K tokens/min
- Anthropic: 50 req/min, 100K tokens/min
- OpenRouter: 200 req/min, 500K tokens/min

システム側対策:

- バックオフ・リトライ
- サーキットブレーカー
- リクエストキューイング

1.10 デプロイメント

1.10.1 本番デプロイ手順

```
# 1. ビルド
cd /ai/prj/techdev
npm run build

# 2. 本番ディレクトリへ同期
sudo rsync -av dist/ /prod/techsapo/dist/
sudo rsync -av node_modules/ /prod/techsapo/node_modules/
sudo rsync -av package.json /prod/techsapo/

# 3. systemd サービス再起動
sudo systemctl restart techsapo

# 4. ステータス確認
sudo systemctl status techsapo

# 5. ヘルスチェック
curl http://localhost:8443/health
```

1.10.2 systemd 設定

場所: /etc/systemd/system/techsapo.service

設定例:

```
[Unit]
Description=TechSapo Wall-Bounce Server
After=network.target

[Service]
Type=simple
User=wombat
WorkingDirectory=/prod/techsapo
ExecStart=/data/.nvm/versions/node/v22.9.0/bin/node dist/index.js
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

1.10.3 監視

ログ監視:

```
# リアルタイムログ
sudo journalctl -u techsapo -f

# エラーのみ
sudo journalctl -u techsapo -p err -f
```

メトリクス:

- CPU 使用率
- メモリ使用量
- レスポンスタイム
- エラーレート

1.10.4 バックアップ

対象:

- 環境変数設定
- Redis データ
- 監査ログ

頻度:

- 日次: Redis スナップショット
- 週次: 監査ログアーカイブ

1.11 まとめ

1.11.1 システムの強み

1. 高品質: 複数 LLM の協調による高精度回答
2. 柔軟性: タスク別最適プロバイダー自動選択
3. 信頼性: コンセンサス・Quorum による品質保証
4. 拡張性: 階層化アーキテクチャで新規プロバイダー追加容易

1.11.2 重要原則 再掲

1. **Anthropic API** 直接使用禁止 明示的有効化時のみ
2. **GPT-5 Codex**: codex exec 経由
3. **Gemini**: gemini コマンド経由
4. 壁打ち: 最低 3 回、最大 6 回
5. コーディング: GPT-5 Codex + Qwen3 Coder 優先

1.11.3 次のステップ

1. 開発環境セットアップ
2. サンプルクエリ実行
3. Wall-Bounce API テスト
4. 独自タスクでの検証

1.11.4 リファレンス

ドキュメント:

- /ai/prj/techdev/docs/ - 全ドキュメント
- /ai/prj/techdev/CLAUDE.md - Claude Code 利用ガイド

リポジトリ:

- /ai/prj/techdev/ - メインプロジェクト

サポート:

- GitHub Issues: 問題報告・機能要望
- ドキュメント: 詳細な技術情報

作成: TechSapo Documentation Team 更新: 2025-10-07 バージョン: 2.0